

Лабораторная работа №2

Rust: владение, ссылки, traits и generics

Аннотация

Дисциплина: программирование на языке Rust

Раздел курса: модель владения и обобщённое программирование

Формат выполнения: индивидуальная лабораторная работа по одному методическому документу

Среды выполнения: Windows + VS Code + rustup, WSL2 (Ubuntu 24.04+), Linux

Лабораторная работа продолжает вводный курс по Rust. Студент создаёт два обязательных Cargo-проекта и на практических примерах осваивает владение, изменяемые ссылки, срезы, traits, обобщённую структуру Pair, Clone и generic-функции.

Оглавление

1	Цель работы	2
2	Правила	2
3	Подготовка рабочей среды	2
3.1	Windows + VS Code + rustup	2
3.2	WSL2 (Ubuntu 24.04+)	2
3.3	Linux	3
4	Как создавать и редактировать файлы	3
5	Создание рабочей структуры лабораторной работы	3
6	Формат сдачи	3
7	Задача 2.1. Список дел и владение данными	3
7.1	Назначение задачи	3
7.2	Шаг 1. Создайте проект	4
7.3	Шаг 2. Запишите файл Cargo.toml	4
7.4	Шаг 3. Запишите файл src/main.rs	4
7.5	Шаг 4. Отформатируйте и проверьте проект	6
7.6	Шаг 5. Запустите программу	6
7.7	Ожидаемый результат	6
7.8	Самопроверка	6
7.9	Типичные ошибки	6
8	Задача 2.2. Traits, generics и учебный отчёт	7
8.1	Назначение задачи	7
8.2	Шаг 1. Создайте проект	7
8.3	Шаг 2. Запишите файл Cargo.toml	7
8.4	Шаг 3. Запишите файл src/main.rs	7
8.5	Шаг 4. Отформатируйте и проверьте проект	9
8.6	Шаг 5. Запустите программу	10
8.7	Ожидаемый результат	10
8.8	Самопроверка	10
8.9	Типичные ошибки	10
9	Мини-шпаргалка по темам работы	10

1 Цель работы

Цель работы состоит в том, чтобы закрепить базовую модель владения Rust на простых, наблюдаемых программах и подготовить студента к чтению более крупных Rust-проектов.

После выполнения работы студент должен уметь:

- объяснять, почему String часто передаётся во владение;
- использовать `&mut Vec<T>` для изменения коллекции;
- использовать `&[T]` для чтения данных без владения;
- описывать методы в блоке `impl`;
- создавать собственный trait и реализовывать его для структуры;
- объявлять обобщённую структуру `Pair<T>`;
- применять `Clone` там, где нужен явный независимый экземпляр значения;
- проверять код через `cargo test`.

2 Правила

Работа состоит из двух обязательных задач. Они выполняются строго по порядку: сначала 2.1, затем 2.2. Пропуск любой задачи не допускается.

В работе используются только стандартная библиотека Rust и стандартный инструмент сборки Cargo. Внешние библиотеки не подключаются, поэтому раздел `[dependencies]` остаётся пустым.

Имена каталогов, файлов и проектов необходимо сохранять без изменений. Все команды ниже предполагают именно эти имена.

3 Подготовка рабочей среды

Если Rust уже установлен после выполнения первой лабораторной работы, повторная установка не требуется. Достаточно проверить инструменты:

```
rustup --version
rustc --version
cargo --version
cargo fmt --version
```

Если Rust ещё не установлен, используйте `rustup`.

3.1 Windows + VS Code + rustup

Откройте официальный сайт <https://www.rust-lang.org/tools/install>, скачайте установщик `rustup-init.exe` и выполните установку. Если установщик предлагает установить Microsoft C++ Build Tools, согласитесь: эти компоненты нужны для сборки программ под Windows.

В VS Code установите расширение `rust-analyzer`. Оно помогает видеть ошибки до запуска компилятора и удобнее перемещаться по коду.

3.2 WSL2 (Ubuntu 24.04+)

В терминале Ubuntu выполните:

```
sudo apt update
sudo apt install -y curl build-essential
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
```

Для работы через VS Code установите расширения WSL и rust-analyzer, затем откройте каталог проекта из терминала Ubuntu командой `code ..`

3.3 Linux

В Debian/Ubuntu используйте:

```
sudo apt update
sudo apt install -y curl build-essential
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
```

В других дистрибутивах установите эквиваленты `curl` и базовых средств сборки, затем выполните установку через `rustup`.

4 Как создавать и редактировать файлы

Каждая задача выполняется как отдельный Cargo-проект. Команда `cargo new --bin --vcs none` создаёт каталог проекта, файл `Cargo.toml` и файл `src/main.rs`. После создания проекта замените содержимое этих файлов на тексты, приведённые ниже.

В VS Code откройте папку лабораторной работы через `File -> Open Folder...`. Панель файлов открывается через `View -> Explorer` или `Ctrl+Shift+E`. Сохраняйте изменения через `Ctrl+S`.

В терминале можно использовать `nano`:

```
nano src/main.rs
```

В `nano` сохранение выполняется через `Ctrl+O`, затем `Enter`; выход — через `Ctrl+X`.

5 Создание рабочей структуры лабораторной работы

В Windows PowerShell:

```
mkdir rust-labs
cd rust-labs
mkdir 02-rust-ownership-traits
cd 02-rust-ownership-traits
```

В WSL2 или Linux:

```
mkdir -p ~/rust-labs/02-rust-ownership-traits
cd ~/rust-labs/02-rust-ownership-traits
```

После этого выполняйте задачи 2.1 и 2.2 строго в указанном порядке.

6 Формат сдачи

На защите необходимо показать два проекта: `task01_todo_ownership` и `task02_traits_generics_report`. Для каждого проекта демонстрируются `Cargo.toml`, `src/main.rs`, успешный запуск `cargo fmt`, успешный запуск `cargo test` и фактический вывод `cargo run --quiet`.

7 Задача 2.1. Список дел и владение данными

Уровень сложности: базовый для темы владения, первая обязательная задача.

7.1 Назначение задачи

Программа создаёт список дел, добавляет несколько задач, отмечает одну задачу выполненной, удаляет одну задачу и печатает итоговое состояние списка. На этой задаче видно, что `String`

передаётся в функцию во владение, `&mut Vec<Task>` позволяет изменять коллекцию, а `&[Task]` подходит для чтения без изменения.

7.2 Шаг 1. Создайте проект

Находясь в каталоге `02-rust-ownership-traits`, выполните:

```
cargo new --bin --vcs none task01_todo_ownership
cd task01_todo_ownership
```

7.3 Шаг 2. Запишите файл `Cargo.toml`

Итоговое содержимое файла `Cargo.toml` должно быть следующим:

```
[package]
name = "task01_todo_ownership"
version = "0.1.0"
edition = "2024"

[dependencies]
```

7.4 Шаг 3. Запишите файл `src/main.rs`

Итоговое содержимое файла `src/main.rs` должно быть следующим:

```
#[derive(Debug, PartialEq, Eq)]
struct Task {
    id: u32,
    title: String,
    completed: bool,
}

impl Task {
    fn new(id: u32, title: String) -> Self {
        Self {
            id,
            title,
            completed: false,
        }
    }

    fn mark_completed(&mut self) {
        self.completed = true;
    }

    fn status_label(&self) -> &'static str {
        if self.completed { "done" } else { "open" }
    }
}

fn add_task(tasks: &mut Vec<Task>, id: u32, title: String) {
    tasks.push(Task::new(id, title));
}

fn complete_task(tasks: &mut [Task], id: u32) -> bool {
    if let Some(task) = tasks.iter_mut().find(|task| task.id == id) {
        task.mark_completed();
        true
    } else {
        false
    }
}
```

```

fn remove_task(tasks: &mut Vec<Task>, id: u32) -> bool {
    if let Some(position) = tasks.iter().position(|task| task.id == id) {
        tasks.remove(position);
        true
    } else {
        false
    }
}

fn count_completed(tasks: &[Task]) -> usize {
    tasks.iter().filter(|task| task.completed).count()
}

fn count_open(tasks: &[Task]) -> usize {
    tasks.len() - count_completed(tasks)
}

fn format_tasks(tasks: &[Task]) -> String {
    tasks
        .iter()
        .map(|task| format!("[{}] {}: {}", task.status_label(), task.id, task.title))
        .collect::()
        .join("\n")
}

fn main() {
    let mut tasks = Vec::new();

    add_task(&mut tasks, 1, String::from("Create Cargo project"));
    add_task(&mut tasks, 2, String::from("Read Rust ownership chapter"));
    add_task(&mut tasks, 3, String::from("Solve borrowing exercise"));

    complete_task(&mut tasks, 2);
    remove_task(&mut tasks, 1);

    println!("Todo list");
    println!("{}", format_tasks(&tasks));
    println!("Completed: {}", count_completed(&tasks));
    println!("Open: {}", count_open(&tasks));
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn adds_task_by_taking_title_ownership() {
        let mut tasks = Vec::new();
        let title = String::from("Learn ownership");

        add_task(&mut tasks, 1, title);

        assert_eq!(tasks.len(), 1);
        assert_eq!(tasks[0].title, "Learn ownership");
    }

    #[test]
    fn completes_existing_task_through_mutable_slice() {
        let mut tasks = vec![
            Task::new(1, String::from("A")),
            Task::new(2, String::from("B")),
        ];
    }
}

```

```

    };

    assert!(complete_task(&mut tasks, 2));
    assert!(tasks[1].completed);
    assert_eq!(count_completed(&tasks), 1);
}

#[test]
fn removes_task_from_vector() {
    let mut tasks = vec![
        Task::new(1, String::from("A")),
        Task::new(2, String::from("B")),
    ];

    assert!(remove_task(&mut tasks, 1));
    assert_eq!(tasks.len(), 1);
    assert_eq!(tasks[0].id, 2);
}
}

```

Структура Task владеет строкой title, потому что поле имеет тип String, а не &str. Функция add_task принимает title: String, то есть получает строку во владение и помещает её внутрь новой задачи. Функции complete_task и remove_task получают изменяемый доступ к коллекции, потому что они меняют её состояние. Функции count_completed, count_open и format_tasks получают только срез &[Task], потому что им достаточно читать данные.

7.5 Шаг 4. Отформатируйте и проверьте проект

```

cargo fmt
cargo test

```

7.6 Шаг 5. Запустите программу

Команда одинакова для Windows PowerShell, WSL2 и Linux:

```

cargo run --quiet

```

7.7 Ожидаемый результат

```

Todo list
[done] 2: Read Rust ownership chapter
[open] 3: Solve borrowing exercise
Completed: 1
Open: 1

```

7.8 Самопроверка

Проверьте, что cargo test показывает три успешных теста. Затем убедитесь, что в выводе нет задачи 1: Create Cargo project: она была удалена функцией remove_task. Задача с номером 2 должна иметь статус done, потому что она была изменена через complete_task.

7.9 Типичные ошибки

Если компилятор сообщает о перемещённом значении, проверьте, не пытаетесь ли вы использовать переменную title после передачи её в add_task. Если возникает ошибка о невозможности изменяемого заимствования, убедитесь, что функция получает &mut Vec<Task> или &mut [Task], а вызов выполняется через &mut tasks. Если вывод пустой, проверьте, что строки add_task находятся до строк complete_task и remove_task.

8 Задача 2.2. Traits, generics и учебный отчёт

Уровень сложности: повышенный для этой работы, вторая обязательная задача.

8.1 Назначение задачи

Во второй задаче создаётся отчёт о результатах студентов. Для хранения пары оценок используется обобщённая структура `Pair<T>`. Для печати строк отчёта объявляется `trait ReportLine`. Затем generic-функция `build_report<T: ReportLine>` строит текст отчёта для любого списка элементов, которые умеют превращаться в строку отчёта.

8.2 Шаг 1. Создайте проект

Вернитесь в каталог `02-rust-ownership-traits` и создайте второй проект:

```
cd ..  
cargo new --bin --vcs none task02_traits_generics_report  
cd task02_traits_generics_report
```

Если вы уже находитесь в каталоге `02-rust-ownership-traits`, первую команду `cd ..` выполнять не нужно.

8.3 Шаг 2. Запишите файл `Cargo.toml`

Итоговое содержимое файла `Cargo.toml` должно быть следующим:

```
[package]  
name = "task02_traits_generics_report"  
version = "0.1.0"  
edition = "2024"  
  
[dependencies]
```

8.4 Шаг 3. Запишите файл `src/main.rs`

Итоговое содержимое файла `src/main.rs` должно быть следующим:

```
#[derive(Clone, Debug, PartialEq, Eq)]  
struct Pair<T> {  
    left: T,  
    right: T,  
}  
  
impl<T> Pair<T> {  
    fn new(left: T, right: T) -> Self {  
        Self { left, right }  
    }  
  
    fn left(&self) -> &T {  
        &self.left  
    }  
  
    fn right(&self) -> &T {  
        &self.right  
    }  
}  
  
impl Pair<u32> {  
    fn sum(&self) -> u32 {  
        self.left + self.right  
    }  
}
```

```

trait ReportLine {
    fn report_line(&self) -> String;
}

#[derive(Clone, Debug, PartialEq, Eq)]
struct StudentResult {
    name: String,
    scores: Pair<u32>,
}

impl StudentResult {
    fn new(name: &str, first_score: u32, second_score: u32) -> Self {
        Self {
            name: name.to_string(),
            scores: Pair::new(first_score, second_score),
        }
    }

    fn total(&self) -> u32 {
        self.scores.sum()
    }
}

impl ReportLine for StudentResult {
    fn report_line(&self) -> String {
        format!(
            "{}: {} + {} = {}",
            self.name,
            self.scores.left(),
            self.scores.right(),
            self.total()
        )
    }
}

fn build_report<T: ReportLine>(items: &[T]) -> String {
    items
        .iter()
        .map(|item| item.report_line())
        .collect::<Vec<_>>()
        .join("\n")
}

fn best_result(items: &[StudentResult]) -> Option<&StudentResult> {
    items.iter().max_by_key(|item| item.total())
}

fn main() {
    let results = vec![
        StudentResult::new("Anna", 42, 48),
        StudentResult::new("Boris", 38, 40),
        StudentResult::new("Olga", 50, 47),
    ];

    println!("Progress report");
    println!("{}", build_report(&results));

    if let Some(best) = best_result(&results) {
        println!("Best: {} with {}", best.name, best.total());
    }
}

```



```

    let copied_sample = results[0].clone();
    println!("Cloned sample: {}", copied_sample.report_line());
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn pair_stores_two_values() {
        let pair = Pair::new(10, 15);

        assert_eq!(*pair.left(), 10);
        assert_eq!(*pair.right(), 15);
        assert_eq!(pair.sum(), 25);
    }

    #[test]
    fn report_line_contains_total_score() {
        let result = StudentResult::new("Anna", 42, 48);

        assert_eq!(result.report_line(), "Anna: 42 + 48 = 90");
    }

    #[test]
    fn generic_report_collects_lines() {
        let results = vec![
            StudentResult::new("Anna", 42, 48),
            StudentResult::new("Boris", 38, 40),
        ];

        let report = build_report(&results);

        assert!(report.contains("Anna: 42 + 48 = 90"));
        assert!(report.contains("Boris: 38 + 40 = 78"));
    }

    #[test]
    fn finds_best_result() {
        let results = vec![
            StudentResult::new("Anna", 42, 48),
            StudentResult::new("Olga", 50, 47),
        ];

        let best = best_result(&results).expect("sample list is not empty");

        assert_eq!(best.name, "Olga");
        assert_eq!(best.total(), 97);
    }
}

```

Структура `Pair<T>` показывает, как один и тот же тип можно использовать для разных значений одного типа. В этой программе отдельно реализован метод `sum` для `Pair<u32>`, потому что складывать нужно именно числовые оценки. `Trait ReportLine` задаёт общее требование: тип должен уметь возвращать строку отчёта. Функция `build_report<T: ReportLine>` работает не с конкретной структурой, а с любым типом, который реализует этот `trait`.

8.5 Шаг 4. Отформатируйте и проверьте проект

```
cargo fmt
cargo test
```

8.6 Шаг 5. Запустите программу

```
cargo run --quiet
```

8.7 Ожидаемый результат

```
Progress report
Anna: 42 + 48 = 90
Boris: 38 + 40 = 78
Olga: 50 + 47 = 97
Best: Olga with 97
Cloned sample: Anna: 42 + 48 = 90
```

8.8 Самопроверка

Проверьте, что `cargo test` показывает четыре успешных теста. В выводе программы студент Olga должен быть лучшим, потому что сумма `50 + 47` равна 97. Последняя строка показывает результат явного копирования значения через `clone`.

8.9 Типичные ошибки

Если компилятор сообщает, что метод `clone` недоступен, проверьте наличие `#[derive(Clone, Debug, PartialEq, Eq)]` у структур `Pair<T>` и `StudentResult`. Если `generic`-функция `build_report` не компилируется, проверьте ограничение `T: ReportLine`. Если в `report_line` возникает ошибка форматирования, убедитесь, что метод возвращает `String`, а не печатает строку напрямую.

9 Мини-шпаргалка по темам работы

Владение означает, что значение принадлежит конкретной переменной. Когда `String` передаётся в функцию без ссылки, владение строкой переходит в эту функцию.

Изменяемое заимствование `&mut` позволяет функции изменить уже существующее значение, не забирая его навсегда.

Срез `&[T]` позволяет читать последовательность элементов без владения всем `Vec<T>`.

Trait описывает набор методов, которые должен предоставить тип.

Generic-тип позволяет написать структуру или функцию не для одного конкретного типа, а для семейства типов.

Clone используется тогда, когда нужен явный независимый экземпляр значения.

10 Мини-шпаргалка по Cargo

- `cargo new --bin --vcs none имя_проекта` — создать новый исполняемый проект.
- `cargo fmt` — отформатировать код.
- `cargo fmt --check` — проверить форматирование без изменения файлов.
- `cargo test` — запустить тесты.
- `cargo run --quiet` — собрать и запустить программу без лишних сообщений Cargo.

11 Финальный чек-лист сдачи

1. Создан каталог 02-rust-ownership-traits.
2. Создан проект task01_todo_ownership.
3. В проекте task01_todo_ownership заполнены Cargo.toml и src/main.rs.
4. Для task01_todo_ownership успешно выполнены cargo fmt, cargo test, cargo run --quiet.
5. Создан проект task02_traits_generics_report.
6. В проекте task02_traits_generics_report заполнены Cargo.toml и src/main.rs.
7. Для task02_traits_generics_report успешно выполнены cargo fmt, cargo test, cargo run --quiet.
8. Оба проекта продемонстрированы преподавателю в установленном порядке: 2.1 -> 2.2.